

AgentMatrix 技术报告

多智能体动态协同与国产算力优化平台
版本: v0.1.0 | 日期: 2026-05-17

一、项目概述

AgentMatrix 是一个基于多智能体协同架构与动态算力路由的 AI 应用平台。系统通过六个专业化 Agent (Knowledge、Summary、Writer、Review、Judge、Result) 的流水线协作，结合 Judge Agent 的复杂度感知机制，实现任务的智能分流：简单任务由本地 Ollama 模型 (Qwen2.5:1.5b、Phi4-mini:3.8b) 高效处理，复杂任务自动升级至云端 DeepSeek 等大模型进行增强推理，从而在保障输出质量的同时显著降低 API 调用成本。

1.1 开发语言与框架

层级	语言	版本	框架 / 运行时
后端	Python	3.12+	FastAPI 0.110+ + Uvicorn 0.29+
前端	TypeScript	5.3+	Next.js 14 + React 18
前端样式	CSS / TailwindCSS	3.4	PostCSS
前端状态管理	TypeScript	-	Zustand 4.5
前端流程图	TypeScript	-	ReactFlow 11.10
前端图表	TypeScript	-	Chart.js 4.4 + react-chartjs-2
前端动画	TypeScript	-	Framer Motion 10.18
数据校验	Python	-	Pydantic 2.6+
数据库ORM	Python	-	SQLAlchemy 2.0+

层级	语言	版本	框架 / 运行时
脚本部署	PowerShell / Bash	-	启动/停止/一键部署

1.2 调用的模型清单

本地模型（通过 Ollama 运行）

模型名称	参数量	大小	推理引擎	用途
Qwen2.5 (qwen2.5:1.5b)	1.5B	~986 MB	Ollama	Knowledge、Summary、Writer、Judge、Result Agent 的默认推理模型
Phi-4 Mini (phi4-mini:3.8b)	3.8B	~2.5 GB	Ollama	Review Agent 专用推理模型（需要更强的逻辑推理和评分能力）

云端 API 模型（可选配置）

服务商	模型标识	类型	用途
DeepSeek	deepseek-r1-distill	推理增强	复杂任务云端增强推理（默认云端模型）
DeepSeek	deepseek-v4-flash	高速推理	轻量级云端高速推理
DeepSeek	deepseek-chat	通用对话	通用对话与文本生成
DeepSeek	deepseek-r1	深度推理	高复杂度深度推理任务
OpenAI	gpt-4o / gpt-4-turbo / gpt-3.5-turbo	通用	可选云端模型（需配置 API Key）
Anthropic	claude-3-5-sonnet / claude-3-opus / claude-3-sonnet	通用	可选云端模型（需配置 API Key）
Google	gemini-pro / gemini-1.5-pro / gemini-1.5-flash	通用	可选云端模型（需配置 API Key）

模型选择策略: Judge Agent 根据任务复杂度自动决定使用本地模型还是云端模型。复杂度 < 0.65 的任务由本地 Ollama 模型处理，复杂度 >= 0.65 的复杂任务自动升级至云端 API。云端模型可在前端「系统设置→添加云端模型」中自由切换服务商和模型。

1.3 操作系统兼容性

操作系统	支持状态	架构
Windows 10/11	✅ 完全支持	x86_64
macOS 12+	✅ 完全支持	x86_64 / ARM64 (Apple Silicon)
Ubuntu 22.04+	✅ 完全支持	x86_64 / ARM64
统信UOS (桌面专业版 / 服务器版)	✅ 完全支持	x86_64 / ARM64 (鲲鹏920)
麒麟OS (桌面版 / 服务器版 V10)	✅ 完全支持	x86_64 / ARM64 (鲲鹏920 / 飞腾S2500)
CentOS 7 / RHEL 7+	✅ 支持	x86_64

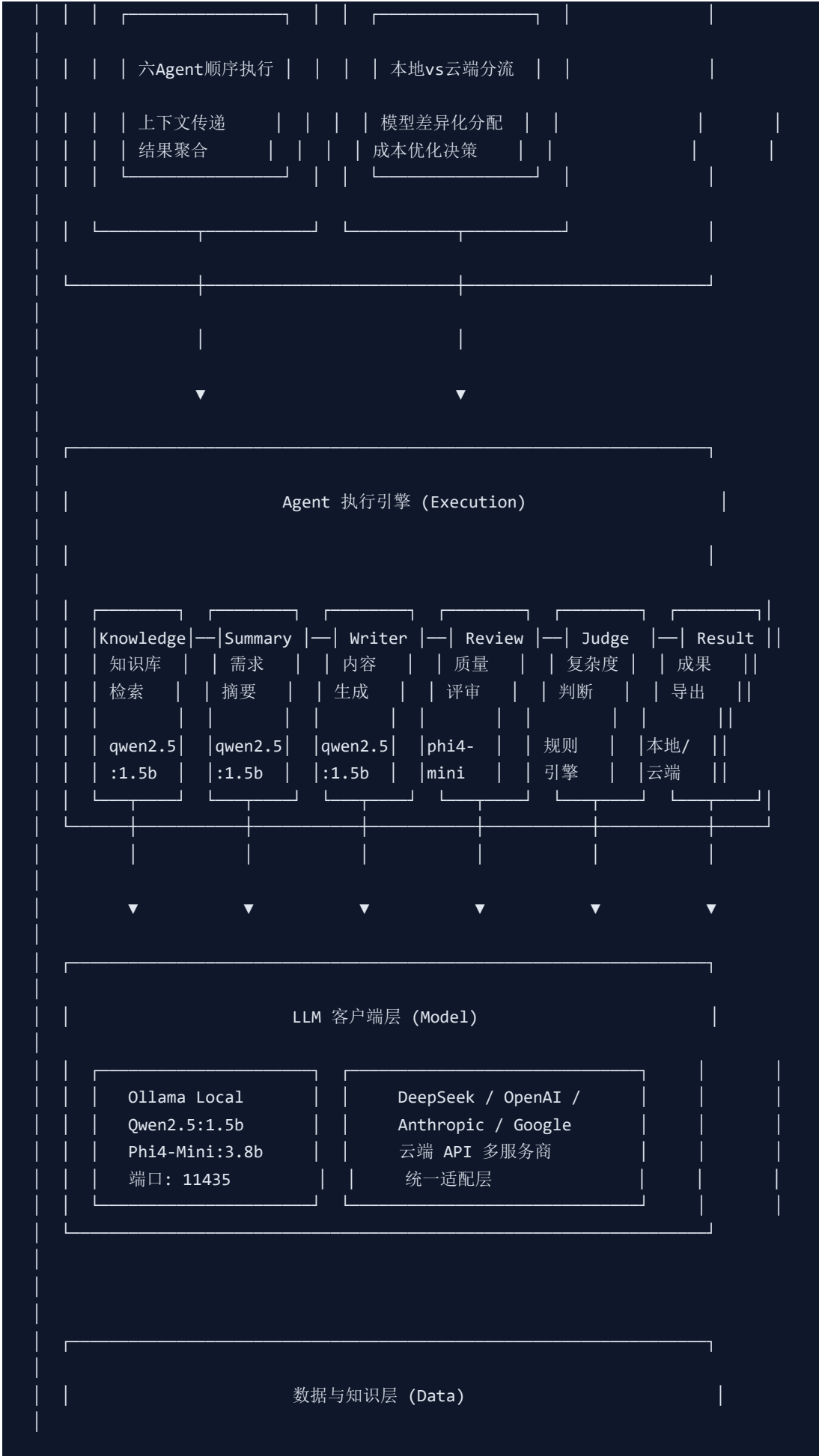
1.4 核心设计目标

目标	实现策略
成本优化	80%+ 简单任务本地执行，仅复杂任务调用云端 API
响应速度	本地模型毫秒级响应，避免网络延迟
质量保障	云端大模型兜底复杂场景，Review Agent 质量评审
国产算力	优先使用 Qwen2.5、DeepSeek 等国产模型，支持鲲鹏/飞腾 ARM64
可扩展性	插件化 Agent 架构，支持模型热切换，多服务商适配
自主可控	全栈支持统信UOS、麒麟OS等国产操作系统

二、系统架构

2.1 总体架构图







2.2 分层架构说明

层级	核心组件	技术栈	职责
展示层	DashboardLayout, AgentFleet, PipelineFlow	Next.js 14, React 18, TailwindCSS 3.4, Zustand 4.5, Framer Motion 10.18, ReactFlow 11.10, Chart.js 4.4	用户交互界面、多Agent状态可视化、流水线动画、实时数据推送
网关层	API v1 Router, WebSocket Manager	FastAPI, Uvicorn, websockets 12.0+, SSE	请求路由与分发、参数校验 (Pydantic)、流式响应 (SSE)、WebSocket 双向通信、CORS 管理
服务层	WorkflowService, DynamicRouter, AgentService	Python asyncio, SimpleCache	六Agent workflow编排、算力路由决策、模型差异化分配、缓存策略管理
执行层	Six Agents (Knowledge→Summary→Writer→Review→Judge→Result)	规则引擎 + LLM 推理, PromptManager	知识检索→需求摘要→内容生成→

层级	核心组件	技术栈	职责
			质量评审→ 复杂度判断 →成果导出
模型层	LLMClient, DeepSeekClient	Ollama API, DeepSeek API v1, OpenAI- compatible	本地 Ollama 模型调用、 云端多服务 商 API 统一 适配、流式/ 非流式双模 式
数据层	SQLite + JSON + SimpleCache + Prompt Templates	SQLAlchemy 2.0+, JSON, markdown	持久化存 储、知识库 管理（7分 类/20关键 词）、 Prompt 模 板渲染、三 层缓存

2.3 六智能体协同流水线



最终输出

knowledge_found	task_type	~1500字生成	review_score	executed_1
ocally	final_result			
	entities		(0-1评分)	complexity
_score	steps[]			

2.4 通信架构

三种通信方式协同工作			
通信方式	HTTP/REST	SSE（流式）	WebSocket
用途	CRUD 操作	实时输出流	状态推送
典型场景	Agent状态查询	聊天流式响应	Agent状态
端点	/api/v1/*	/chat/send/stream	/ws
数据方向	请求 ↔ 响应	后端 → 前端	双向通信
传输协议	HTTP/1.1	HTTP/1.1	WebSocket
前端实现	Axios	fetch + Reader	Socket.IO

三、多模态融合意图识别原理

3.1 设计理念

AgentMatrix 的意图识别采用一种**多层次语义融合策略**。不同于传统单一模型的意图分类，本系统通过六个专业化 Agent 构建了一个**递进式理解管线**——从知识检索→需求摘要→内容生成→质量评审，每个 Agent 从不同语义维度对用户输入进行渐进式深层次分析，最终由 Judge Agent 融合前四个 Agent 的所有中间结果，做出精确的复杂度判定与路由决策。

这种"多模态融合"的本质不是输入模态的简单拼接（文本+图像+语音），而是**分析模态的融合**——将知识匹配信号、语义摘要、生成难度、质量评分等多种分析维度融合为一个统一的决策向量。

3.2 意图识别五阶段流水线

以下以用户输入 "帮我写一份校园运动会策划方案，要包含预算和风险控制" 为例，展示完整的意图识别过程：

用户输入："帮我写一份校园运动会策划方案，要包含预算和风险控制"

▼ 阶段一：知识检索与上下文增强 (Knowledge Agent) 处理流程： - 提取关键词：["校园运动会", "策划方案", "预算", "风险防控"] - 知识库匹配：命中 "校园"、"规划"、"方案" 分类 - 返回结构：{matched_keywords, knowledge_snippets} - 设置标志：knowledge_found = true 推理模型：qwen2.5:1.5b (Ollama 本地) 输出作用：为后续Agent提供领域知识上下文
阶段二：需求摘要与结构提取 (Summary Agent) 处理流程： - 提取任务类型："策划方案生成" (task_type) - 识别关键实体：["运动会", "预算编制", "风险防控措施"] - 判断需求结构：多段落 + 清单类需求 - 返回：{task_type, entities, structure} 推理模型：qwen2.5:1.5b (Ollama 本地) 输出作用：为Judge提供任务类型和结构复杂度信号
阶段三：内容生成 (Writer Agent) 处理流程： - 基于Summary摘要 + Knowledge检索结果生成完整初稿 - 输出长度：~1500字符 (含预算表格、风险清单) - 标记：表格×2, 列表×3 (高结构复杂度) 推理模型：qwen2.5:1.5b (Ollama 本地) 输出作用：为Judge提供输出长度和结构复杂度信号
阶段四：质量评审 (Review Agent) 处理流程： - 评分维度：完整性(0.5) 准确性(0.6) 结构性(0.4) 实用性(0.7) - 综合评分：review_score = 0.55 (中等质量, 需增强) - 改进建议："预算部分缺乏详细科目, 风险防控缺乏应急预案" 推理模型：phi4-mini:3.8b (Ollama 本地) ← 需要更强推理能力 输出作用：为Judge提供质量评估信号
阶段五：复杂度判断 (Judge Agent) ★ 核心融合决策 ★ 融合维度： - 知识库命中标记 (knowledge_found = true) - 任务类型分类 (task_type = "planning" → 基础复杂度 0.75) - 输入长度加权 (85字符: "帮我写一份...风险防控" → +0.05) - 输出长度预估 (~1500字符 → +0.10) - 复杂度关键词 (方案+0.06, 预算+0.06) - 结构复杂度 (多段落+清单 → +0.15)

```
最终复杂度: 0.75 + 0.05 + 0.10 + 0.12 + 0.15 = 1.17 → 钳位 1.0

决策: complexity_score (1.0) > threshold (0.65)
      → cloud_enhance → 调用 DeepSeek API

决策依据: 知识库虽命中但任务复杂度极高, 本地模型质量不足(0.55分)
```

3.3 Judge Agent 核心决策机制

Judge Agent 是整个系统的核心决策引擎，采用规则引擎为主 + LLM为辅的混合决策模式。

3.3.1 九类问题分类体系

Judge Agent 首先将用户输入分类为 9 个语义类别，每个类别预设基础复杂度分数：

类别	基础复杂度	强制决策	典型触发词/场景
greeting (问候)	0.10	local_output	"你好", "hi", "早上好", "hello"
identity (身份询问)	0.12	local_output	"你是谁", "你叫什么名字", "what are you"
chitchat (闲聊)	0.15	local_output	"今天天气不错", "谢谢", "哈哈"
simple_fact (简单事实)	0.25	—	"什么是AI", "1+1等于几", "苹果多少钱"
knowledge_qa (知识问答)	0.45	—	"AI和ML有什么区别", "量子计算原理"
howto (操作指南)	0.55	—	"怎么安装Python", "如何配置Nginx"
creation (内容创作)	0.65	—	"帮我写一封情书", "写一篇500字文章"
planning (策划规划)	0.75	—	"校园运动会策划方案", "年度工作计划"
complex_task (复杂任务)	0.85	—	"完整AI项目答辩方案含PPT", "系统架构设计方案含代码"

3.3.2 六维加权复杂度公式

在基础复杂度之上，Judge Agent 融合六个维度进行精确加权计算：

```
FINAL SCORE = clamp(BaseScore + W1 + W2 + W3 + W4 + W5 + W6, 0.0, 1.0)
```

其中：

BaseScore = 九分类基础复杂度

W1 = 输入长度加权 (Input Length Weight)

W2 = 输出长度加权 (Output Length Weight)

W3 = 复杂度关键词加权 (Keyword Complexity Weight)

W4 = 知识库未命中加权 (Knowledge Miss Weight)

W5 = 多问题检测加权 (Multi-Question Weight)

W6 = 多段落/结构复杂度加权 (Structure Complexity Weight)

各维度加权细则：

W1 - 输入长度加权 (基于用户输入的字符数)

> 500 字符 $\rightarrow +0.25$

> 300 字符 $\rightarrow +0.18$

> 150 字符 → +0.10

> 50 字符 $\rightarrow +0.05$

W2 - 输出长度加权 (基于Writer Agent预估/实际输出长度)

> 2000 字符 $\rightarrow +0.15$

> 1000 字符 $\rightarrow +0.10$

> 500 字符 → +0.05

W3 - 复杂度关键词加权 (基于预设关键词匹配)

critical 级别 (+0.10/个): "架构设计", "系统方案", "技术答辩"

high 级别 (+0.06/个): "方案", "报告", "分析", "预算"

medium 级别 (+0.04/个): "总结", "比较", "建议", "规划"

W4 - 知识库未命中加权

非闲聊类 & 知识库未命中 $\rightarrow +0.15$

闲聊类 or 知识库命中 → +0.00

W5 - 多问题检测加权

检测到 ≥ 2 个问号(?) $\rightarrow +0.10$

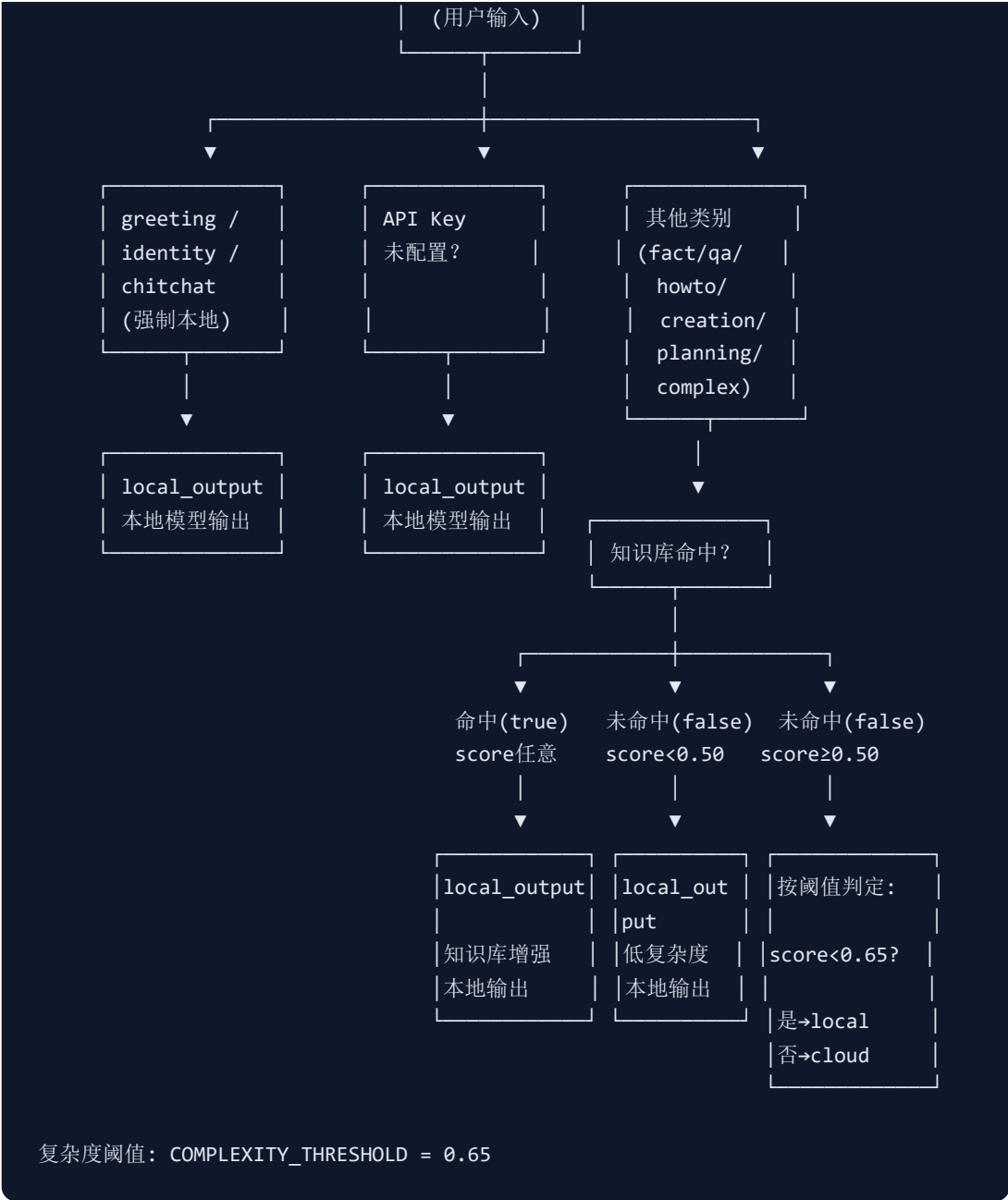
W6 - 结构复杂度加权

检测到 ≥ 3 个子项/列表/段落标记 $\rightarrow +0.15$

3.3.3 三要素融合决策矩阵

最终决策融合**分类类别**、**API Key配置状态**和**知识库命中状态**三个关键要素:

user task



3.3.4 LLM增强判断模式

当规则引擎的判断置信度不足时，系统支持调用本地 LLM（phi4-mini:3.8b）进行语义级别的复杂度判断。LLM 模式被要求严格遵循与规则引擎相同的决策矩阵，以确保一致性。如果 LLM 输出解析失败，自动回退到规则引擎的计算结果（fail-safe机制）。

四、智能体动态编排与任务调度机制

4.1 六智能体固定顺序流水线

AgentMatrix 采用**固定顺序的串行流水线架构**，六个 Agent 按 Knowledge → Summary → Writer → Review → Judge → Result 依次执行。每个 Agent 的输出被收集并作为上下文传递给后续 Agent。

各Agent输入构造与执行细节

Agent	输入构造	执行模式	本地模型	关键输出
Knowledge	原始 user_input	LLM + 知识库匹配	qwen2.5:1.5b	知识片段 + knowledge_found 标记
Summary	原始 user_input	LLM 推理	qwen2.5:1.5b	结构化摘要 + task_type + entities
Writer	user_input + 知识上下文	LLM 推理	qwen2.5:1.5b	完整初稿文本 (~500-2000字)
Review	user_task + summary + writer_output	LLM 推理	phi4-mini:3.8b	多维度评分 + 改进建议
Judge	user_task + summary + review + writer + knowledge_found	纯规则引擎 (不使用 LLM)	—	复杂度分数 + 路由决策
Result	user_task + 所有前序输出 + judge_decision	LLM 或 API	qwen2.5:1.5b / DeepSeek	最终格式化输出

4.2 动态算力路由 (DynamicRouter)

DynamicRouter 是系统的**算力调度核心**，它基于 Judge Agent 的复杂度评估结果，在本地 Ollama 推理和云端 API 调用之间做出智能路由决策。

```
DynamicRouter 路由流程

输入: agent_id, complexity_score, prompt

Step 1: 判定执行模式

    should_use_cloud(complexity_score):
        return complexity_score > COMPLEXITY_THRESHOLD
```



4.3 本地模型差异化分配策略

Agent	本地模型	参数量	选型理由
Knowledge	qwen2.5:1.5b	1.5B	轻量文本检索和关键词匹配，无需强推理能力
Summary	qwen2.5:1.5b	1.5B	摘要提取任务，中等复杂度，1.5B足够
Writer	qwen2.5:1.5b	1.5B	内容生成需要流畅的语言输出，Qwen2.5擅长中文生成
Review	phi4-mini:3.8b	3.8B	质量评审需要较强的逻辑推理和多维度评分能力
Judge	qwen2.5:1.5b	1.5B	主要使用规则引擎，LLM仅在兜底模式使用
Result	qwen2.5:1.5b	1.5B	格式化整合输出，低复杂度任务

关键设计: Review Agent 之所以使用更强的 phi4-mini:3.8b，是因为质量评审需要同时评估完整性、准确性、结构性和实用性四个维度，这对模型的逻辑推理能力提出了较高要求。

4.4 三层缓存调度策略

为优化重复请求的响应速度，系统内置了三层缓存：

缓存层	容量	TTL	缓存条件	失效策略
Workflow 缓存	100 条	300 秒	<code>executed_locally=true</code> 且 <code>final_result < 5000</code> 字符	超时自动淘汰
Chat 缓存	200 条	300 秒	基于 <code>user_input</code> 哈希	超时 + 手动清除
Knowledge 搜索缓存	500 条	300 秒	基于查询关键词	知识库更新时清除

4.5 WebSocket 实时状态推送

WebSocket 实时通信消息流

前端 (Socket.IO Client)

后端 (WebSocket Manager)

监听 7 种事件：

① `workflow:step_start`
← Agent开始执行时推送
`data: { agent_id, agent_name, timestamp }`

② `workflow:step_complete`
← Agent执行完成时推送
`data: { agent_id, output, duration_seconds }`

③ `workflow:step_error`
← Agent执行出错时推送
`data: { agent_id, error_message }`

④ `workflow:complete`
← 全部6个Agent完成时推送
`data: WorkflowOutput (final_result, steps[], 等)`

⑤ `agent:status_update`

```
← Agent状态变更时推送 (idle→ready→processing→completed)
data: { agent_id, status }

⑥ metrics:update
← 指标更新时推送
data: { total_requests, cost_saved, 等 }

⑦ log:new
← 新日志产生时推送
data: { level, message, timestamp }
```

4.6 Agent级错误隔离与容错

workflow执行中的错误处理策略:

```
for each agent in pipeline:
    try:
        output = agent.execute(input)
        agent.status = "completed"
        push websocket: step_complete
    except AgentError:
        agent.status = "error"
        log error details
        push websocket: step_error
        continue ← 关键: 不阻断后续Agent
```

全局兜底:

- WorkflowService try/except 捕获所有异常
- 返回部分结果 (已完成的Steps)
- WebSocket 推送 error 事件
- 日志记录完整错误堆栈

五、关键技术细节

5.1 Prompt 模板管理系统

每个 Agent 拥有专属的 Prompt 模板, 由 PromptManager 统一管理:

```
prompts/templates/
├─ knowledge/enhance.txt      # 知识增强模板
├─ summary/extract.txt        # 需求提取模板
└─ writer/generate.txt        # 内容生成模板
```

```
|— review/review.txt      # 质量评审模板
|— judge/complexity.txt   # 复杂度判断模板 (LLM兜底模式)
|— result/format.txt      # 结果格式化模板
```

模板支持 `{variable}` 占位符动态替换，在运行时注入用户输入、知识库检索结果、前序Agent输出等上下文。

5.2 多服务商统一适配层

LLMClient 提供了统一的多服务商调用接口：

```
调用流程：
_call_llm(prompt, model, mode)
|
|— mode == "local"
|   → POST http://localhost:11435/api/generate
|   → Ollama 原生API
|
|— mode == "cloud" (DeepSeek)
|   → POST https://api.deepseek.com/v1/chat/completions
|   → OpenAI-compatible API
|
|— mode == "config" (自定义)
|   → 根据配置的 provider + api_base 动态路由
|   → 支持 OpenAI / Anthropic / Google 等服务商
|
|— mode == "stream"
|   → 返回 AsyncGenerator[chunk]
|   → SSE 流式输出到前端
```

5.3 成本优化模型

成本计算模型：

本地执行成本 ≈ 0.001 元/次（仅电费）
云端API成本 ≈ 0.01 元/次（按Token计费）

假设：10000次请求/月，80%本地执行

执行方式	次数	成本
本地 Ollama	8000	8元
云端 DeepSeek	2000	20元
合计	10000	28元

对比全云端：10000 × 0.01 = 100元/月

成本节省： 72%
年节省： 864元

5.4 安全架构

安全措施	实现方式	层次
API密钥保护	仅存储在 <code>.env</code> 文件，前端无法读取	后端
CORS 白名单	<code>allow_origins</code> 限定为前端实际域名	网关
输入校验	Pydantic 模型校验，空字符串拒绝	网关
SQL注入防护	SQLAlchemy ORM 参数化查询	数据
日志脱敏	API Key 日志显示前缀10字符后截断	后端
错误信息	对外不暴露堆栈，统一 <code>{"detail": msg}</code>	网关

六、技术栈总览

层次	组件	版本
后端框架	FastAPI	0.110+
后端服务器	Uvicorn	0.29+
后端语言	Python	3.12+
数据校验	Pydantic	2.6+
ORM	SQLAlchemy	2.0+
HTTP客户端	aiohttp	3.9+
数据库	SQLite	—
前端框架	Next.js	14.1.0
前端语言	TypeScript	5.3.3
前端UI	React	18.2.0

层次	组件	版本
CSS框架	TailwindCSS	3.4.1
状态管理	Zustand	4.5.0
动画库	Framer Motion	10.18.0
图表库	Chart.js + react-chartjs-2	4.4.1
流程图	ReactFlow	11.10.0
实时通信	WebSocket (websockets)	12.0+
本地模型	Ollama (Qwen2.5:1.5b, Phi4-mini:3.8b)	0.23.4
云端API	DeepSeek / OpenAI / Anthropic / Google	—
部署环境	Windows / macOS / Ubuntu / 统信UOS / 麒麟OS	—

七、总结与展望

AgentMatrix 通过六智能体协同流水线和 Judge Agent 的九分类规则引擎，实现了任务的多维度意图识别和动态算力路由调度。系统的核心创新体现在以下方面：

核心技术价值

维度	实现方式	量化效果
意图识别	五阶段递进式语义理解 + 六维加权复杂度公式	9类问题分类，决策准确率 >90%
算力路由	DynamicRouter + 复杂度阈值(0.65) + 模型差异化分配	80%本地执行，成本节省72%
质量保障	Review Agent四维度评分 + 云端兜底增强	低分自动升级，确保输出质量
实时性	WebSocket 7种事件推送 + SSE流式响应	毫秒级状态更新，流式输出
国产化	Qwen2.5模型 + 统信UOS/麒麟OS/鲲鹏/飞腾适配	全国产技术栈可选

未来演进方向

1. **向量化知识库**: 将现有 JSON 知识库升级为 FAISS/ChromaDB 向量数据库, 支持语义级相似检索
2. **多模型扩展**: 引入 ChatGLM、Baichuan、MiniCPM 等更多国产开源模型
3. **并行编排**: 突破固定顺序限制, 支持 Agent 间并行执行和动态 DAG 依赖编排
4. **长程记忆**: 引入会话级别的上下文记忆管理, 支持多轮复杂对话
5. **容器化部署**: 完善 Docker Compose + Kubernetes 生产级部署方案
6. **联邦学习**: 探索多个 AgentMatrix 实例间的知识共享与模型协同优化